



UNIVERSIDADE
DE VIGO

ESCUELA SUPERIOR
DE INGENIERÍA INFORMÁTICA

Grado en Inteligencia Artificial

Practica 2 SIEX

Sistemas CBR: valoración de vulnerabilidades

Raúl Vázquez Arévalo

Curso 2025-2026

INDICE

1. Descripción del problema.....	3
1.1. Contexto del dominio.....	3
1.2. Objetivos del Sistema.....	3
1.3 Solucion propuesta.....	3
2. Descripción de la implementación.....	4
2.1. Métricas de similaridad empleadas.....	4
Atributos y Funciones de Similaridad Local.....	4
Ponderación (Pesos utilizados).....	5
2.2. Detalles de implementación de las fases del ciclo CBR.....	6
Fase 1: RECUPERAR.....	6
Fase 2: REUTILIZAR.....	6
Fase 3: REVISAR.....	6
Fase 4: RETENER.....	6
2.3 Implementación de los niveles de severidad.....	7
3. Ejemplo de funcionamiento y descripción de las pruebas realizadas.....	8
3.1. Ejecución del sistema.....	8
Sintaxis y Parámetros.....	8
Ejemplo de Traza de Ejecución.....	9
3.2. Descripción de las pruebas realizadas.....	10
Interpretación de los resultados.....	12
4. Conclusiones y problemas encontrados.....	13
4.1. Idoneidad de la herramienta y la metodología.....	13
4.2. Conclusiones del análisis experimental.....	13
4.3. Problemas encontrados.....	14
4.4. Posibles mejoras.....	15
5. Bibliografía consultada.....	15

1. Descripción del problema

El objetivo principal de este proyecto es el desarrollo de un prototipo de herramienta para la valoración automática de informes de vulnerabilidad de software. En el ámbito de la ciberseguridad, el volumen de nuevas vulnerabilidades descubiertas crece constantemente, lo que hace necesaria la asistencia de sistemas inteligentes para su clasificación y evaluación inicial.

El problema se aborda utilizando un **Sistema de Razonamiento Basado en Casos (CBR)**. La premisa fundamental es utilizar la experiencia acumulada en registros históricos de vulnerabilidades para inferir las características de nuevas vulnerabilidades detectadas.

1.1. Contexto del dominio

El sistema trabaja con tres estándares fundamentales de la industria de la seguridad informática:

- **CVE (Common Vulnerabilities and Exposures):** Es el sistema de referencia para identificar y clasificar vulnerabilidades. Cada registro incluye datos como un identificador único, una descripción textual del problema, los productos afectados y referencias externas .
- **CWE (Common Weakness Enumeration):** Es una jerarquía que clasifica los tipos de debilidades de software. Cada CVE está asociado a un tipo de debilidad CWE, lo que proporciona una categorización conceptual de la vulnerabilidad.
- **CVSS (Common Vulnerability Scoring System):** Es el marco estandarizado para evaluar la gravedad de las vulnerabilidades. Define métricas base que representan las características intrínsecas de la vulnerabilidad.

1.2. Objetivos del Sistema

El sistema CBR diseñado debe procesar registros **CVE simplificados** y predecir dos atributos críticos pertenecientes a las métricas CVSS:

1. **score:** Un valor numérico entre 0 y 10 que indica la peligrosidad de la vulnerabilidad (problema de regresión).
2. **attackVector:** La forma en que un atacante puede explotar la vulnerabilidad (problema de clasificación). Los valores posibles son NETWORK, ADJACENT_NETWORK o LOCAL.

Para lograr esto, el sistema debe recuperar casos similares de una base de casos histórica basándose en la similitud de atributos como la descripción textual, el tipo de debilidad (CWE), los productos afectados y la entidad asignadora. Posteriormente, debe adaptar estas soluciones recuperadas para proponer una valoración para el nuevo caso, revisar su corrección basándose en reglas de niveles de severidad CVSS y decidir si retener el nuevo conocimiento aprendido.

1.3 Solucion propuesta

Para la solución del problema se usarán los datos proporcionados en la explicación de la práctica, el miniframework `core.py` copiado del ejemplo del tasador y se implementaron `main_cve.py` y `revisor_cve.py` siguiendo la estructura de dicho ejemplo colgado en moovi.

2. Descripción de la implementación

El sistema ha sido desarrollado en Python (versión 3.12) extendiendo la clase base CBR proporcionada en el miniframework `core.py`. Para la gestión de datos y el cálculo de similaridades complejas, se ha integrado la librería CBRkit (v0.25.8).

2.1. Métricas de similitud empleadas

La fase de recuperación se fundamenta en una métrica de similitud global compuesta, definida mediante `cbrkit.sim.attribute_value`. Esta métrica agrega las similaridades locales de los distintos atributos del registro CVE mediante una media ponderada.

Se han implementado dos modos de funcionamiento para el tratamiento del texto, seleccionables comentando y descomentando líneas en el código, para demostrar la diferencia entre precisión semántica y velocidad de cómputo.

Atributos y Funciones de Similitud Local

1. `cwe` (Taxonomía)

- Tipo de dato: Cadena (ID de la jerarquía CWE).
- Métrica: Se utiliza la métrica de Wu-Palmer sobre la estructura jerárquica proporcionada en `jerarquia_cwe_1000.yaml`. Esta métrica evalúa la distancia semántica entre dos conceptos basándose en su profundidad y en el nodo ancestro común más cercano.
- Implementación: `cbrkit.sim.taxonomy.build(taxonomia_cwe, cbrkit.sim.taxonomy.wu_palmer())`

2. `affected_products` (Colección)

- Tipo de dato: Lista de cadenas.
- Métrica: Se emplea una métrica de conjuntos para comparar las listas. Dado que los nombres de productos pueden tener variaciones menores, se utiliza Isolated Mapping.
- Métrica interna: Para comparar los elementos individuales de la lista, se utiliza la distancia de Levenshtein, permitiendo coincidencias aproximadas entre nombres de software.
- Implementación: `cbrkit.sim.collections.isolated_mapping` con `element_similarity=cbrkit.sim.strings.levenshtein`.

3. `assigner` (Categoría)

- Tipo de dato: Cadena simple.
- Métrica: Igualdad estricta.
- Implementación: `cbrkit.sim.generic.equality()`.

4. Tratamiento de Texto (Dualidad description vs keywords)

- **Por defecto - Atributo description:** Se utiliza similitud semántica basada en modelos de lenguaje (LLM). Se generan embeddings vectoriales del texto usando el modelo all-MiniLM-L6-v2 de Sentence Transformers y se comparan mediante la similitud del Coseno.
 - Implementación: `cbrkit.sim.embed.build(`
`conversion_func=embedding_sin_cache,`
`sim_func=cbrkit.sim.embed.cosine())`
 - Optimización: Para mitigar la latencia, se utiliza una caché de vectores persistente (`cbrkit.sim.embed.cache`) sobre una base de datos SQLite. De no ser así, el tiempo de ejecución sería inabordable.
- **Alternativa - Atributo keywords:** Como alternativa de alto rendimiento, se utiliza el atributo keywords (lista de términos extraídos). Se aplica el coeficiente de Jaccard, que mide la intersección sobre la unión de los conjuntos de palabras clave. Esta alternativa sí que presenta un tiempo de ejecución más abordable.
 - Implementación: `cbrkit.sim.collections.jaccard()`

Ponderación (Pesos utilizados)

La agregación final se realiza mediante una media ponderada (`pooling="mean"`) con la siguiente distribución de pesos, diseñada para priorizar la descripción del problema y su categorización técnica:

Atributo	Peso	Justificación
Texto (description o keywords)	0.4	Contiene la información más rica y específica de la vulnerabilidad.
cwe	0.3	Define el tipo conceptual de error, crucial para determinar la severidad.
affected_products	0.2	El tipo de software afectado influye en el vector de ataque y el impacto.
assigner	0.1	La entidad reportadora tiene una relevancia menor pero aporta contexto.

2.2. Detalles de implementación de las fases del ciclo CBR

Fase 1: RECUPERAR

Se utiliza los módulos `cbrkit.sim` y `cbrkit.retrieval`.

- `Cbrkit.sim` se usa masivamente dentro del método `inicializar_retriever`. Su función es definir las reglas de cómo se comparan los casos (taxonomías, embeddings, levenshtein, etc.). Sin este módulo, el sistema no sabría qué casos se parecen entre sí.

- `Cbrkit.retrieval` se usa al final de `inicializar_retriever` (para empaquetar esas reglas en un objeto "buscador") y dentro del método `recuperar` (para ejecutar la búsqueda real usando esas reglas). Se basa en aplicar la métrica de similaridad descrita anteriormente sobre la base de casos. El sistema recupera los *k* casos más similares (configurable, por defecto *k*=5), utilizando un filtro `cbrkit.retrieval.dropout`.

Fase 2: REUTILIZAR

El sistema predice los valores para el nuevo caso basándose en los *k* vecinos recuperados:

- Predicción de score (Regresión): Se calcula la media ponderada de los scores de los casos recuperados, utilizando sus valores de similaridad como pesos.
- Predicción de `attackVector` (Clasificación): Se calcula la moda (valor más frecuente) entre los vectores de ataque de los casos recuperados.

Fase 3: REVISAR

Se simula una revisión experta comparando los valores predichos con los valores reales. Se aplican las reglas de negocio definidas por CVSS v3.1:

1. Verificación de score: Se considera correcta si el valor predicho y el real pertenecen al mismo nivel de severidad (None, Low, Medium, High, Critical).
2. Verificación de `attackVector`: Se considera correcta solo si hay coincidencia exacta.
3. Determinación de Éxito: Un caso se marca como éxito si el score es correcto, o si el `attackVector` es correcto y el score predicho cae en un nivel de severidad adyacente al real.
4. Corrección: Si una predicción no cumple los criterios de corrección, el sistema sobrescribe el valor predicho con el valor real del caso.

Fase 4: RETENER

El sistema decide si incorporar el nuevo caso a la base de conocimiento siguiendo dos criterios excluyentes:

1. Casos Corregidos: Si el sistema falló en la predicción y tuvo que corregir algún valor, se retiene el caso con los valores reales (corregidos) para aprender del error.

2. Casos de Éxito Total: Si el sistema predijo correctamente tanto el score como el attackVector, se retiene el caso con los valores predichos (que coinciden con los reales) para reforzar el conocimiento existente

2.3 Implementación de los niveles de severidad

Para realizar la práctica se consideran una serie de predicciones, donde para considerarlas correctas o erróneas (algunas, no todas) se piden unos rangos de niveles de severidad de la v3.1 de CVSS para encuadrar los valores reales y los predichos. Dichos valores y rangos posibles son:

None (0.0), Low (0.1-3.9), Medium (4.0-6.9), High (7.0-8.9), Critical (9.0-10.0)

A continuación se explica dicha implementación en el fichero `revisor_cve.py`, dividido en 3 funciones:

- `_get_severity_level()`
- `_get_severity_numeric_level()`
- `_are_severity_numeric_level()`

```
class RevisorCBR(CBR):  
  
    #niveles de severidad  
    def _get_severity_level(self, score):  
        if score == 0.0: return "None"  
        elif 0.1 <= score <= 3.9: return "Low"  
        elif 4.0 <= score <= 6.9: return "Medium"  
        elif 7.0 <= score <= 8.9: return "High"  
        elif 9.0 <= score <= 10.0: return "Critical"  
        return "Unknown"  
  
    #aplicar un número a los niveles de severidad  
    def _get_severity_numeric_level(self, score):  
        level = self._get_severity_level(score)  
        mapping = {"None": 0, "Low": 1, "Medium": 2, "High": 3, "Critical": 4, "Unknown": -1}  
        return mapping.get(level, -1)  
  
    #calcular los niveles de severidad adyacentes  
    def _are_severity_levels_adjacent(self, score1, score2):  
        level1_num = self._get_severity_numeric_level(score1)  
        level2_num = self._get_severity_numeric_level(score2)  
        if level1_num == -1 or level2_num == -1: return False  
        return abs(level1_num - level2_num) == 1
```

- Con respecto a `_get_severity_level()`, esta función se encarga de asignar dichos rangos/valores a los nombres que les corresponde:

- Si `score == 0.0` se devuelve `None`
- Si `score` está entre 0.1 y 3.9 se devuelve `Low`
- Si `score` está entre 4.0 y 6.9 se devuelve `Medium`
- Si `score` está entre 7.0 y 8.9 se devuelve `High`
- Si `score` está entre 9.0 y 10.0 se devuelve `Critical`
- Si no sucede ninguno de los anteriores casos se devuelve `Unknown`

- En relación a `_get_severity_numeric_level()`, esta función se encarga de transformar dichos nombres de rangos/valores de la función anterior a números para poder realizar comparaciones entre ellos (en concreto calcular rangos adyacentes):

- Si se devolvió `None`, la función lo convierte en 0
- Si se devolvió `Low`, la función lo convierte en 1
- Si se devolvió `Medium`, la función lo convierte en 2
- Si se devolvió `High`, la función lo convierte en 3
- Si se devolvió `Critical`, la función devuelve en 4
- Si se devolvió `Unknown`, la función devuelve -1

- Por último, nos centramos en `_are_severity_levels_adjacent()`, una función que se utiliza para calcular cuales son los rangos adyacentes, aspecto que se pide para calcular los casos de éxito.

- Primero se le pasan como argumentos el score real y el score predicho
- Se calculan los números asociados a cada uno de los rangos a los que pertenece cada score
- Si alguno de esos números es -1 (“Unknown”) el sistema devuelve `False` (no adyacentes)
- Se calcula la resta de los números de los rangos de los dos scores en valor absoluto. Si dicha operación es igual a 1, los niveles de severidad de los scores son adyacentes.

3. Ejemplo de funcionamiento y descripción de las pruebas realizadas

3.1. Ejecución del sistema

El sistema está diseñado para ejecutarse desde la línea de comandos mediante el script principal `main_cve.py`. Este enfoque permite automatizar pruebas y modificar la configuración del ciclo CBR mediante argumentos sin necesidad de alterar el código fuente.

Sintaxis y Parámetros

Para ejecutar el programa, se utiliza el intérprete de Python invocando el script principal. La sintaxis general es:

`python main_cve.py [opciones]`

Los parámetros disponibles para configurar la ejecución son los siguientes:

- `-b / --base_casos`: Ruta al archivo JSON con la base de casos (Por defecto: `datos/cve.base_casos.json`).
- `-r / --casos`: Ruta al archivo JSON con los casos a resolver (Por defecto: `datos/cve.casos_a_resolver.json`).
- `-c / --cwe`: Ruta a la taxonomía CWE en formato YAML.
- `-e / --cache`: Ruta a la base de datos SQLite con la caché de embeddings.
- `-n / --num_similares`: (Entero) Número de vecinos más cercanos (k) que se recuperarán en la fase de recuperación. Por defecto es 5.

- -d / --debug: (Flag) Activa el modo detallado. Imprime por consola el paso a paso del ciclo CBR (casos recuperados, similitudes, valores intermedios de predicción y decisiones de revisión).

Ejemplo de Trazas de Ejecución

A continuación, se presenta la traza real de ejecución para uno de los casos, obtenida utilizando Sentence Transformers y k=5.

En este ejemplo se observa cómo el sistema predice un Score de 6.5 (Nivel Medium) frente al real de 7.7 (Nivel High). Aunque los valores numéricos no coinciden, el caso se considera un ÉXITO porque:

1. El vector de ataque (NETWORK) se predijo correctamente.
2. El nivel de severidad predicho (Medium) es adyacente al real (High), cumpliendo la regla de relajación definida en el sistema.

Sin embargo, al no ser una coincidencia exacta en el score, el sistema marca el caso como corregido: True y lo retiene con los valores reales.

```
--- Procesando Caso 100/100 (ID: CVE-2022-35978) ---
[DEBUG.recuperar] CASO A RESOLVER: ID: CVE-2022-35978, Score: 7.7, AV: NETWORK -> [META: id: CVE-2022-35978, score_real: 7.7, score_pred: 0.0, av_real: NETWORK, av_pred: UNKNOWN, exito: False, corregido: False]
[DEBUG.recuperar] CASOS SIMILARES (total: 5, similitud max.:0.7293315569840477)
[DEBUG.recuperar] - [1] ID: CVE-2020-15174, Score: 7.5, AV: NETWORK (Similitud: 0.7293315569840477)
[DEBUG.recuperar] - [2] ID: CVE-2020-15215, Score: 5.6, AV: NETWORK (Similitud: 0.7290041480385578)
[DEBUG.recuperar] - [3] ID: CVE-2022-39266, Score: 9.7, AV: ADJACENT_NETWORK (Similitud: 0.7190170215250052)
[DEBUG.recuperar] - [4] ID: CVE-2022-36085, Score: 7.4, AV: NETWORK (Similitud: 0.7024626468827918)
[DEBUG.recuperar] - [5] ID: CVE-2021-32729, Score: 2.0, AV: NETWORK (Similitud: 0.6661387891745455)
[DEBUG.recuperar]
[DEBUG.reutilizar] CASO RESUELTO: ID: CVE-2022-35978, Score: 6.5, AV: NETWORK -> [META: id: CVE-2022-35978, score_real: 7.7, score_pred: 6.5, av_real: NETWORK, av_pred: NETWORK, exito: False, corregido: False]
[DEBUG.reutilizar]
[DEBUG.revisar] CASO REVISADO: ID: CVE-2022-35978, Score: 7.7, AV: NETWORK -> [META: id: CVE-2022-35978, score_real: 7.7, score_pred: 6.5, av_real: NETWORK, av_pred: NETWORK, exito: True, corregido: True]
[DEBUG.revisar] - exito: True
[DEBUG.revisar] - corregido: True
[DEBUG.revisar]
[DEBUG.retener] CASO RETENIDO: ID: CVE-2022-35978, Score: 7.7, AV: NETWORK -> [META: id: CVE-2022-35978, score_real: 7.7, score_pred: 6.5, av_real: NETWORK, av_pred: NETWORK, exito: True, corregido: True]
[DEBUG.retener] - retenido: True con id manual: CVE-2022-35978
[DEBUG.retener]
--- CASO RESUELTO ---
{ '_meta': { 'attackVector_correcto': True,
            'attackVector_predicho': 'NETWORK',
            'attackVector_real': 'NETWORK',
            'corregido': True,
            'exito': True,
            'id': 'CVE-2022-35978',
            'score_correcto': False,
            'score_predicho': 6.502446202951492,
            'score_real': 7.7},
  'affected_products': ['minetest::minetest'],
  'assigner': 'GitHub_M',
  'cwe': 'CWE-693',
```

```
'description': 'Minetest is a free open-source voxel game engine...',  
'id': 'CVE-2022-35978',  
'keywords': [ 'the game session', 'mod', 'easy modding', ...],  
'metric': {'attackVector': 'NETWORK', 'score': 7.7},  
'title': 'Lua sandbox escape from mod in Minetest'}  
-----
```

3.2. Descripción de las pruebas realizadas

Para validar la viabilidad técnica del sistema, se realizó un análisis del rendimiento centrado en el cuello de botella computacional más crítico identificado en la documentación: el cálculo de similitud semántica sobre el atributo description.

Los pesos utilizados siguen siendo los mencionados en la distribución inicial.

Se evaluaron tres escenarios de ejecución distintos para contrastar tiempos de respuesta y calidad de las predicciones:

1. Escenario de referencia: Sentence Transformers sin caché Se realizó una prueba desactivando la carga de la base de datos de vectores precalculados para forzar la inferencia del modelo all-MiniLM-L6-v2 en tiempo real sobre la CPU.

- **Observación:** El tiempo de procesamiento por caso se disparó drásticamente. La proyección de tiempo para procesar el lote completo de 100 casos se estimó en **aproximadamente 10 horas**.
- **Conclusión:** Se confirma la advertencia de la documentación sobre la inviabilidad de utilizar modelos de lenguaje profundos (LLMs) en un sistema CBR de este tipo sin estrategias de optimización o hardware dedicado (GPU) .

2. Escenario optimizado: Sentence Transformers con caché SQLite Se activó el uso de la caché de embeddings (embeddings_cache.db) utilizando el decorador cbrkit.sim.embed.cache.

- **Rendimiento temporal:** El tiempo de ejecución se redujo de horas a segundos, eliminando la latencia de inferencia.
- **Calidad de la predicción:** Se obtuvo una tasa de éxito del **85.00%** (85/100 casos).
- **Conclusión:** La caché permite obtener los beneficios de la precisión semántica de los Transformers sin penalización de tiempo.

3. Escenario alternativo: Keywords (Jaccard) Se utilizó el modo rápido (keywords) que emplea el coeficiente de Jaccard sobre listas de palabras clave, una operación computacionalmente ligera.

- **Rendimiento temporal:** El tiempo de ejecución fue equivalente al del escenario con caché, demostrando ser una alternativa muy eficiente.
- **Calidad de la predicción:** Se obtuvo una tasa de éxito del **84.00%** (84/100 casos).
- **Conclusión:** La diferencia de calidad entre el uso de modelos complejos (Transformers) y métodos estadísticos simples (Keywords) fue marginal (solo un 1% de diferencia) en este conjunto de datos, lo que valida el uso de keywords como una estrategia de respaldo robusta y rápida.

Análisis de sensibilidad de los pesos

Adicionalmente, se realizaron tres pruebas de configuración modificando los pesos de la función de agregación (pooling_weights) para identificar qué atributos aportan mayor valor predictivo al sistema.

1. Prueba "Enfoque Estructural" (Prioridad a CWE) Se aumentó el peso del atributo cwe a **0.7**, reduciendo el resto.

- **Resultados:**

- Acierto en Score: **60%**
- Acierto en Attack Vector: **80%**
- Tasa de éxito global: **84%**
- **Análisis:** Esta configuración logró el **mejor rendimiento en la predicción del Score** de entre estas tres pruebas de sensibilidad (60%). Esto confirma que la severidad de una vulnerabilidad está fuertemente vinculada a su naturaleza técnica (tipo de error CWE). Sin embargo, al no priorizar el producto o el texto, el acierto en el vector de ataque se mantuvo en un 80%, lo que limitó la tasa de éxito global al 84%.

2. Prueba "Enfoque de Producto" (Prioridad a Affected Products) Se aumentó el peso del atributo affected_products a **0.7**.

- **Resultados:**

- Acierto en Score: **55%**
- Acierto en Attack Vector: **81%**
- Tasa de éxito global: **87%**
- **Análisis:** Se observó una mejora en la clasificación del vector de ataque (81%), lo que impulsó la tasa de éxito global al 87%. Esto valida la hipótesis de que la superficie de ataque depende del software afectado. Como contrapartida, la precisión del Score cayó al 55% (la más baja de las tres pruebas), evidenciando que conocer el producto no es suficiente para determinar la gravedad exacta del fallo, ya que un mismo software puede tener vulnerabilidades de distinta criticidad.

3. Prueba "Ciega al Texto" (Sin Description) Se eliminó la influencia del texto (description = 0.0), repartiendo el peso entre cwe (0.5), affected_products (0.4) y assigner (0.1).

- **Resultados:**

- Acierto en Score: **56%**
- Acierto en Attack Vector: **81%**
- Tasa de éxito global: **87%**

- **Análisis:** A pesar de prescindir de la descripción semántica, el sistema mantuvo un rendimiento sobresaliente (87% de éxito). El acierto en Score (56%) se mantuvo competitivo gracias al peso del CWE, y el acierto en Attack Vector (81%) fue alto gracias a la influencia del producto. Este resultado es muy revelador: sugiere que, para este conjunto de datos, la combinación de metadatos estructurados (Tipo + Producto) es prácticamente tan eficaz como el análisis de texto profundo, permitiendo construir modelos más ligeros sin gran pérdida de precisión.

→ Para determinar la correlación directa de cada atributo con la solución final, se realizó una batería de pruebas asignando un peso unitario (1.0) a cada atributo de forma aislada, anulando la influencia del resto.

Los resultados, presentados en la siguiente tabla resumen, arrojan conclusiones muy significativas sobre la naturaleza de los datos CVE:

Atributo Aislado (Peso 1.0)	Acierto Score (Severidad)	Acierto Attack Vector (Exacto)	Éxito Global
Keywords	61.0%	84.0%	89.0%
Affected Products	40.0%	81.0%	88.0%
Description (Embeddings)	56.0%	79.0%	81.0%
CWE (Taxonomía)	41.0%	79.0%	83.0%
Assigner	43.0%	75.0%	83.0%

Interpretación de los resultados

1. La supremacía de las Keywords (Jaccard) El hallazgo más notable es que el atributo keywords ofrece el mejor rendimiento en todas las métricas, superando incluso a la descripción semántica profunda (description con embeddings).

- **Análisis:** Con un **61% de acierto en Score** y un **84% en el vector de ataque**, se demuestra que la extracción de términos específicos es más determinante para clasificar una vulnerabilidad que el significado semántico global de la frase. La métrica de Jaccard captura estas coincidencias exactas de vocabulario técnico de forma muy eficaz.

2. El comportamiento dual de Affected Products Esta prueba valida perfectamente la hipótesis sobre la influencia del software en el vector de ataque.

- **Análisis:** Obtuvo un excelente **81% en Attack Vector** (2º mejor), confirmando que saber qué producto es define casi unívocamente cómo se ataca (NETWORK vs LOCAL). Sin embargo, obtuvo el peor resultado en **Score (40%)**, lo cual es lógico: un mismo producto puede tener vulnerabilidades críticas y leves indistintamente.
- **Nota:** Su alta tasa de éxito global (88%) se debe a la Regla 4 del sistema, que concede el éxito si el vector es correcto y el score es simplemente adyacente (algo fácil de conseguir por azar si el vector ya es correcto).

3. Description vs. Keywords Aunque el modelo de Sentence Transformers (description) tuvo un buen desempeño en la predicción del Score (56%), fue superado por el método de keywords.

- **Análisis:** Esto sugiere que, para el dominio específico de CVEs, la precisión léxica (palabras exactas) pesa más que la semántica difusa. Los embeddings pueden diluir la importancia de una palabra técnica clave dentro de una frase larga, mientras que Jaccard la penaliza o premia directamente.

4. El rol de CWE y Assigner

- **CWE:** Sorprendentemente, el tipo de vulnerabilidad por sí solo no fue un gran predictor del Score (41%), sugiriendo que un mismo tipo de error puede tener impactos muy variables según el contexto.
- **Assigner:** Aunque se esperaba un rendimiento bajo, logró un 75% en Vector de Ataque. Esto probablemente se debe a que ciertos fabricantes (Assigners) reportan casi exclusivamente vulnerabilidades de sus propios productos web o de escritorio, creando una correlación indirecta con el vector de ataque.

Conclusión del experimento: La combinación de atributos es necesaria para robustez, pero el atributo keywords ha demostrado ser el predictor individual más potente en todos los aspectos.

4. Conclusiones y problemas encontrados

4.1. Idoneidad de la herramienta y la metodología

El desarrollo de este proyecto ha validado que el Razonamiento Basado en Casos (CBR) es una metodología idónea para la valoración automática de vulnerabilidades. El problema se ajusta naturalmente al paradigma CBR: las nuevas vulnerabilidades suelen ser variaciones de problemas conocidos. Se ha comprobado que patrones como "tipo de error CWE específico en un producto de software concreto" tienden a replicar la misma severidad y vector de ataque, lo que permite predecir valores basándose en la analogía histórica sin necesidad de reglas "if-then" complejas.

La utilización de la librería **CBRkit** ha resultado fundamental. Su diseño modular ha permitido integrar métricas de similitud heterogéneas (taxonomías para CWE, conjuntos para palabras clave y listas para productos) en un único recuperador semántico, facilitando una implementación limpia que cumple con los requisitos del ciclo CBR estándar.

4.2. Conclusiones del análisis experimental

Tras un extenso análisis de sensibilidad y rendimiento, se extraen tres conclusiones fundamentales que definen el comportamiento del sistema:

1. Viabilidad técnica y coste computacional Se confirmó que el uso de modelos de lenguaje profundos (Sentence Transformers) introduce una latencia inasumible en tiempo real (estimada en horas para el lote de prueba) si se ejecutan sobre CPU. Sin embargo, la implementación de la **caché**

de vectores SQLite demostró ser una gran solución técnica, reduciendo los tiempos de respuesta a segundos y haciendo viable el uso de similitud semántica avanzada.

2. Jerarquía de poder predictivo (Análisis individual) Las pruebas de aislamiento (peso 1.0 por atributo) revelaron qué datos dictan realmente la solución:

- **Keywords (Jaccard):** Se erigió como el predictor más potente, logrando la mayor tasa de éxito individual (**89%**), el mayor acierto en Score (**61%**) y el mayor acierto en attackVector (**84%**).
- **Affected Products:** Demostró ser crítico para determinar el vector de ataque (**81% de acierto exacto**), validando que la superficie de ataque depende del software, aunque es ineficaz para predecir la gravedad por sí solo (40% de acierto en Score).
- **CWE:** Correcta determinación del vector de ataque pero pobre con respecto a la tasa de éxito del Score.

3. Simplicidad vs. Complejidad El hallazgo más relevante del proyecto surgió al comparar el modelo simple ("Solo Keywords") contra un modelo híbrido teóricamente optimizado (Keywords (0.5) + Products (0.3) + CWE (0.2)).

- El modelo híbrido obtuvo una tasa de éxito del **87%**, inferior al **89%** logrado por el modelo de keywords aislado.
- **Conclusión:** La introducción de atributos adicionales, aunque semánticamente relevantes, introdujo "ruido" que diluyó la precisión del mejor predictor. Esto demuestra que, para este conjunto de datos específico, la coincidencia léxica exacta de términos técnicos (capturada por Jaccard en keywords) es superior a la combinación de múltiples fuentes de información. En este dominio, el modelo más simple resultó ser también el más eficaz.

4.3. Problemas encontrados

Durante el desarrollo se identificaron los siguientes desafíos:

- **Cuello de botella computacional:** El cálculo de similitud entre descripciones textuales utilizando el modelo all-MiniLM-L6-v2 es intensivo en CPU. Sin la utilización de la caché proporcionada en embeddings_cache.db, la fase de recuperación se vuelve el paso limitante del ciclo, confirmando las advertencias de la documentación sobre la lentitud de estos métodos en tiempo real.
- **Ajuste manual de pesos:** Definir la importancia relativa de cada atributo en la función de agregación (pooling_weights) es una tarea compleja. Al no existir un método automático en el prototipo, los pesos se ajustaron empíricamente. Esto puede llevar a configuraciones que no sean óptimas donde se infravalora información importante.
- **Ambigüedad en la predicción numérica:** La predicción del score mediante media ponderada es efectiva para estimar la gravedad general, pero difícilmente logra predecir el valor decimal exacto (ej. predecir 7.4 cuando el real es 7.5). Por ello, el criterio de éxito

basado en "niveles de severidad" resultó ser una métrica de evaluación mucho más realista que el error cuadrático medio.

4.4. Posibles mejoras

Para evolucionar este prototipo hacia un sistema de producción, se proponen las siguientes mejoras:

1. **Optimización automática de pesos:** Implementar un algoritmo de aprendizaje que ajuste automáticamente los pesos de similitud basándose en un conjunto de validación, eliminando la subjetividad del ajuste manual.
2. **Revisión interactiva:** En el ciclo actual, la fase de revisión es automática y supervisada. En un entorno real, el sistema debería incluir un mecanismo de confianza: si la similitud del caso recuperado es baja, el sistema debería solicitar validación a un experto humano antes de retener el caso.
3. **Enriquecimiento de datos:** Incorporar más atributos del estándar CVSS v3.1 a la predicción, como la "Complejidad del ataque" o los "Privilegios requeridos", para ofrecer una valoración de seguridad más completa.

5. Bibliografía consultada

Para el desarrollo de esta práctica y la comprensión del dominio del problema, se han consultado las siguientes fuentes de información y documentación técnica:

1. Guion de la Práctica:

- Práctica 2. Sistema de Razonamiento Basado en Casos (CBE) para valoración de vulnerabilidades. [SIEX_practica2.pdf](#).

2. Estándares de Ciberseguridad:

- **CVE Program:** Common Vulnerabilities and Exposures (CVE). Información sobre el formato y estructura de los registros de vulnerabilidades. Disponible en: https://en.wikipedia.org/wiki/Common_Vulnerabilities_and_Exposures
<https://www.cve.org/>
- **MITRE:** Common Weakness Enumeration (CWE). Jerarquía de tipos de debilidades de software utilizada para la taxonomía del sistema. Disponible en: https://en.wikipedia.org/wiki/Common_Weakness_Enumeration
<https://cwe.mitre.org/>
- **FIRST.org:** Common Vulnerability Scoring System (CVSS v3.1: Specification Document). Especificación de las métricas base (Score y Vector de Ataque) utilizadas para la predicción. Disponible en: https://en.wikipedia.org/wiki/Common_Vulnerability_Scoring_System
<https://nvd.nist.gov/vuln-metrics/cvss>

3. Librerías y Recursos Técnicos:

- **CBRkit Documentation:** Documentación oficial de la librería Python utilizada para la implementación del ciclo CBR y las métricas de similaridad. [resumen_cbrkit.pdf](#)
- **SentenceTransformers Documentation:** Información técnica sobre el modelo de lenguaje all-MiniLM-L6-v2 utilizado para la generación de embeddings de las descripciones. Disponible en: <https://www.sbert.net/>
- **Archivos a utilizar en la practica:** <https://github.com/CVEProject/cvelistV5>